

Chương 03 - Vòng Đời Ứng Dụng (Application's Life Cycle)

1. Cấu trúc ứng dụng Android (Anatomy of Android Applications)

Các thành phần cốt lõi (Core components) là những khối xây dựng cơ bản của ứng dụng. Mỗi thành phần có một vòng đời (lifecycle) độc lập quy định cách nó được tạo ra, chuyển đổi và tiêu hủy. Có 4 loại chính:

- **Activities (Hoạt động):**
 - Cung cấp giao diện người dùng (GUI) và tương tác trực tiếp với người dùng.
 - Mỗi ứng dụng có một Activity chính làm điểm bắt đầu (entry point).
 - Chuyển giao quyền điều khiển và dữ liệu sang Activity khác thông qua **Intents**.
- **Services (Dịch vụ):**
 - Chạy ngầm (background) và không có giao diện người dùng.
 - Thực hiện các tác vụ kéo dài (VD: phát nhạc, lấy vị trí GPS).
- **Broadcast Receivers (Bộ thu phát sóng):**
 - Lắng nghe và phản hồi các tin nhắn/sự kiện từ hệ thống hoặc ứng dụng khác (VD: pin yếu, có wifi).
 - Sử dụng bộ lọc (filter) để chỉ nhận các tin nhắn khớp. Không có giao diện nhưng có thể gọi Activity hoặc dùng Notification.
- **Content Providers (Nhà cung cấp nội dung):**
 - Quản lý và chia sẻ dữ liệu (VD: danh bạ, hình ảnh, SQLite) giữa các ứng dụng một cách an toàn thông qua các phương thức tiêu chuẩn (query, insert, update, delete).

2. Quản lý tiến trình và Ngăn xếp Activity (Activity Stack)

- Mỗi ứng dụng chạy trong một máy ảo (VM) riêng biệt.
- **Tiêu diệt tiến trình:** Hệ điều hành Android có thể buộc dừng bất kỳ ứng dụng nào nếu thiếu bộ nhớ (RAM). Quyết định dựa trên:
 - Trạng thái hiện tại của ứng dụng.
 - Độ quan trọng đối với người dùng (đang tương tác hay chạy ngầm).
- **Activity Stack (Ngăn xếp Hoạt động):**
 - Khi một Activity mới mở ra, nó được đặt lên **đỉnh** của ngăn xếp (Foreground) và chiếm quyền điều khiển.
 - Khi người dùng nhấn nút **Back**, Activity hiện tại bị hủy bỏ khỏi đỉnh ngăn xếp, Activity bên dưới sẽ trồi lên và hoạt động trở lại.

3. Các trạng thái của Activity (Activity States)

Có 3 trạng thái chính:

1. **Running / Active (Đang chạy):** Nằm trên cùng của ngăn xếp, hiển thị toàn màn hình và có tiêu điểm (focus) nhận tương tác.
2. **Paused (Tạm dừng):** Mất tiêu điểm nhưng vẫn còn hiển thị (VD: bị che bởi một popup hoặc Activity trong suốt). Vẫn giữ nguyên trạng thái nhưng có thể bị OS đóng nếu hệ thống cạn kiệt RAM.

3. **Stopped (Đã dừng)**: Bị che khuất hoàn toàn, không còn hiển thị (invisible). Vẫn giữ trạng thái nhưng rất dễ bị OS dọn dẹp để giải phóng bộ nhớ.

4. Các hàm Callback trong Vòng đời Activity (Life Cycle Callbacks)

OS thông báo sự thay đổi trạng thái qua các hàm `protected` :

- **`onCreate()`** : (**Bắt buộc**) Gọi 1 lần duy nhất khi tạo Activity. Dùng để khởi tạo giao diện (`setContentView()`), kết nối các nút bấm, thiết lập sự kiện.
- **`onStart()`** : Activity chuẩn bị hiển thị với người dùng.
- **`onResume()`** : Activity hiển thị, nằm trên cùng và bắt đầu tương tác.
- **`onPause()`** : (**Khuyến dùng**) Gọi khi Activity bắt đầu bị nhường chỗ. Thường dùng để **lưu trữ dữ liệu quan trọng** hoặc tạm dừng các tác vụ tốn tài nguyên. *Là hàm duy nhất được đảm bảo sẽ chạy trước khi bị OS kill.*
- **`onStop()`** : Activity hoàn toàn không còn hiển thị.
- **`onDestroy()`** : Activity chuẩn bị bị hủy và dọn dẹp khỏi bộ nhớ.

(Chu trình hiển thị đầy đủ: `onCreate` → `onStart` → `onResume` → (Đang chạy) → `onPause` → `onStop` → `onDestroy`)

5. Lưu trữ trạng thái và Dữ liệu (State & Data Persistence)

- **SharedPreferences**:
 - Cơ chế lưu trữ dưới dạng cặp `<key, value>` vĩnh viễn (persistent). Giống `HashMap` nhưng lưu vào bộ nhớ trong.
 - Rất thích hợp để lưu trạng thái nhỏ (VD: cài đặt màu sắc, điểm số) trong hàm **`onPause()`** để tránh mất dữ liệu khi OS tắt ứng dụng.
 - Đường dẫn vật lý: `data/data/your-package-name/shared-prefs` .
- **Sử dụng Bundle (`onSaveInstanceState()`)**:
 - Dùng để lưu trạng thái thông qua Bundle `outState.putString(...)` .
 - **Lưu ý quan trọng**: Rất tốt khi xử lý việc OS tự đóng app (VD: **xoay màn hình**), nhưng **không** hoạt động nếu người dùng chủ động tắt app (nhấn nút Back). Vì vậy, dùng `SharedPreferences` ở `onPause()` là cách làm an toàn và ưu việt hơn.

6. Xử lý sự kiện xoay màn hình (Device Orientation)

- Khi thiết bị xoay ngang/dọc, Activity mặc định sẽ bị Destroy và Recreate (Tạo lại từ đầu).
- Lấy hướng thiết bị: Dùng (`WindowManager`)
`getApplication().getSystemService(Context.WINDOW_SERVICE).getDefaultDisplay().getRotation()` .
- Có thể lấy góc xoay ban đầu ở `onCreate` và so sánh góc xoay mới ở `onPause` để biết thiết bị có đang bị xoay hay không.